TinyMonitor Web
Page to API to Helper to Payload

CBM Vale

8 March 2026

Objective

This document explains each page with a technical backend flow:

```
URL / HTML Page -> View / API -> Helper Function -> Payload
```

For each API, the deck explains:

- request arguments,
- process inside the API and helper,
- returned payload fields.

1. Page Map

Page URL	Page view	API count
`/`	`index()`	2 APIs
`/dashboard/`	`dashboard()`	2 APIs
`/kpi/`	`kpi()`	2 APIs
`/kpi_units/`	`kpi_units()`	2 APIs
`/kpi/update_manualdata`	`kpi_manualdata()`	no separate API
`/advisory/`	`advisory()`	1 API
`/advisory/chart`	`advisory_chart()`	1 API
`/settings/`	`settings_page()`	1 API

2. Home Page Overview

Page flow

```
/ -> pages.views.index(request) -> render(request, "home/index.html", {})
```

APIs called by the page

- /api/timeinfo_detail
- /api/severity_plot

3. Home API 1: `timeinfo_detail()`

Flow

`/ -> /api/timeinfo_detail -> apis.views.timeinfo_detail() -> helper_fun.get_TimeInformastion()`

Arguments

No request arguments are read.

Process

The helper reads the latest row from `db/severity_trendings.db`, table `original_sensor`, then builds:

- latest timestamp,
- next update time = latest timestamp + 10 minutes.

Return

`datetime_last, next_update`

4. Home API 2: `severity_plot()`

Flow

`/ -> /api/severity_plot -> apis.views.severity_plot() -> helper_fun.get_SeverityNLoss(start_date, end_date)`

Arguments

`start_date`, `end_date`

Process

The helper:

- adjusts the date range with `get_FixedDate()`,
- reads latest severity and confidence,
- selects the top abnormal features,

5. Home API 2: Process and Return

Flow continuation

```
helper_fun.get_SeverityNLoss(start_date, end_date)
```

Process continued

- loads actual sensor, prediction mean, prediction std, and loss,
- prepares only the top features for plotting.

Return

`counter_feature_s2`, `df_timestamp`, `df_feature_send`, `y_pred_send`, `y_pred_std_send`, `loss_send`, `thr_now_model`

6. Dashboard Page Overview

Page flow

`/dashboard/ -> pages.views.dashboard(request) -> render(request, "home/dashboard.html", {})`

APIs called by the page

- `/api/panel_summary`
- `/api/zone_distribution`

7. Dashboard API 1: `panel_summary()`

Flow

`/dashboard/ -> /api/panel_summary -> apis.views.panel_summary() -> helper_fun.get_PanelSummary(start_date, end_date)`

Arguments

`start_date`, `end_date`

Process

The helper:

- fixes the requested date range,
- reads original sensor data, additional sensor data, and severity trends,
- converts severity percentages into severity levels,

8. Dashboard API 1: Process and Return

Flow continuation

```
helper_fun.get_PanelSummary(start_date, end_date)
```

Process continued

- builds latest value maps and historical value maps,
- counts severity frequencies per feature,
- ranks the most severe features,
- computes `priority_parameter` for each feature.

Return

`last_timestamp`, `last_sensor_featname`, `sensor_featname`, `last_severity_featname`, `sever_featname`, `ordered_feature_name`, `sever_count_featname`, `priority_parameter`

9. Dashboard API 2: `zone_distribution()`

Flow

`/dashboard/ -> /api/zone_distribution -> apis.views.zone_distribution() -> helper_fun.get_OperationDistribution(start_date, end_date, tags)`

Arguments

`start_date`, `end_date`, optional `tags`

Process

The API parses `tags` from comma-separated text into a list. If tags are missing, the helper defaults to `['LGS1']`. Then the helper reads KPI timeline data and calculates:

- operation mode distribution,
- operation zone distribution.

Return

`operation_mode, operation_zone`

10. KPI Overview Page Overview

Page flow

```
/kpi/ -> pages.views.kpi(request) -> render(request, "kpi/kpi.html", {})
```

APIs called by the page

- `/api/unitStatus`
- `/api/kpi`

11. KPI API 1: `unitStatus()`

Flow

`/kpi/ -> /api/unitStatus -> apis.views.unitStatus() -> helper_fun.get_units_status(start_date, end_date, tags)`

Arguments

`start_date`, `end_date`

Process

The API uses a hardcoded tag list: `LGS1, LGS2, LGS3, BGS1, BGS2, KGS1, KGS2`. The helper loops over units and determines whether the latest load code means:

- `alive`, or
- `shutdown`.

Return

`status_dict`

12. KPI API 2: `kpi()`

Flow

`/kpi/ -> /api/kpi -> apis.views.kpi() -> helper_fun.get_KPIData(start_date, end_date, tags, noe_metric)`

Arguments

`start_date`, `end_date`, optional `tags`, optional `noe_metric`

Process

The helper:

- reads KPI tables per unit,
- prepares OEE-related series,
- builds plant aggregate series,

13. KPI API 2: Process and Return

Flow continuation

`helper_fun.get_KPIData(start_date, end_date, tags, noe_metric)`

Process continued

- loads manual event recap from `number_of_event.pickle`,
- optionally loads `other_kpis.pickle`,
- returns one combined payload for units, plant, and manual KPI blocks.

Return overview

Top-level keys include per-unit entries plus `plant`, `noe`, and `other_kpi`.

14. KPI API 2 Return Structure

Top-level keys

Per-unit keys such as `LGS1`, `LGS2`, `BGS1`, plus `plant`, `noe`, and `other_kpi`

Per-unit return fields

`timestamp`, `oee`, `phy_avail`, `performance`, `uo_Avail`, `aux_0`, `aux_1`

Extra payloads

`plant`: aggregate power and plant series
`noe`: grouped event metrics
`other_kpi`: `MTBF`, `MTTR`, `TEEP`

15. KPI Unit Page Overview

Page flow

`/kpi_units/ -> pages.views.kpi_units(request) -> render(request, "kpi/kpi_units.html", {})`

APIs called by the page

- `/api/zone_distributionTimeline`
- `/api/kpi`

16. KPI Unit API 1: `zone_distributionTimeline()`

Flow

`/kpi_units/ -> /api/zone_distributionTimeline ->`
`apis.views.zone_distributionTimeline() ->`
`helper_fun.get_OperationDistributionTimeline(start_date, end_date, tags)`

Arguments

`start_date`, `end_date`, optional `tags`

Process

The helper reads one unit timeline table from `db/kpi.db`, creates a temporary DataFrame, labels each row by load class, converts the load label to a load code, and returns timeline-ready arrays.

Return

`data_timestamp`, `load_datas`, `grid_datas`

17. KPI Unit API 2: `kpi()`

Flow

`/kpi_units/ -> /api/kpi -> apis.views.kpi() -> helper_fun.get_KPIData(start_date, end_date, tags, noe_metric)`

Arguments

Same as KPI overview page: `start_date`, `end_date`, optional `tags`, optional `noe_metric`

Process and return

The same KPI helper is reused here, but now the frontend typically uses it together with one selected unit timeline from `zone_distributionTimeline()`.

18. Manual Upload Page Overview

Page flow

`/kpi/update_manualdata -> pages.views.kpi_manualdata(request)`

Special case

This page does not call a separate `/api/*` endpoint. Its POST processing is done directly inside the page view.

19. Manual Upload POST Flow

Arguments

`request.method`, uploaded `file`, and `request.POST['type_data']`

Process

If `type_data = noe`, the view reads Excel, derives `Start` and `End`, and writes `db/number_of_event.pickle`.
If `type_data = other_kpi`, the view reads Excel, converts month and week into date ranges, and writes `db/other_kpis.pickle`.

Return

Success: `{'message': 'File read successfully', 'type': 'success'}`
Failure: `{'message': 'Error reading file: ...', 'type': 'danger'}`

20. Advisory Table Page Overview

Page flow

```
/advisory/ -> pages.views.advisory(request) -> render(request,
"advisory/advisory.html", {})
```

API called by the page

```
/api/advisory_table
```

21. Advisory API: `advisory_table()`

Flow

`/advisory/ -> /api/advisory_table -> apis.views.advisory_table() -> helper_fun.get_advisoryTable(start_date, end_date)`

Arguments

`start_date`, `end_date`

Process

The helper reads severity trending data, applies smoothing and one feature adjustment, computes priority values, builds one-week severity history, counts severity levels, and orders features by latest severity.

Return

`last_timestamp`, `last_severity_featname`, `sever_1week_featname`, `sever_count_featname`, `severity_counter_overyear`, `priority_parameter`

22. Advisory Detail Page Overview

Page flow

```
/advisory/chart -> pages.views.advisory_chart(request) -> render(request,
"advisory/advisory_chart.html", {})
```

API called by the page

```
/api/advisory_detail/<feat_id>
```

23. Advisory Detail API: Arguments and Process

Flow

`/advisory/chart -> /api/advisory_detail/<feat_id> -> apis.views.advisory_detail() -> helper_fun.get_advisoryDetail(...)`

Arguments

`start_date`, `end_date`, path `feat_id`, `feat_correlate`, `feat_correlate_addi`, `maximum_points`

Process

The API parses correlation lists from text into integer arrays. The helper then:

- selects requested columns,
- reads severity, original sensor, and additional sensor tables,
- computes shutdown periods,

24. Advisory Detail API: Process and Return

Flow continuation

```
helper_fun.get_advisoryDetail(...)
```

Process continued

- smooths the selected severity trend,
- builds correlation-ready sensor arrays,
- computes priority for the selected feature.

Return summary

The endpoint returns timestamps, severity trend, signal values, correlation payloads, shutdown periods, and priority.

25. Advisory Detail API: Return Fields

Payload fields

`feat_id`, `data_timestamp`, `data_timestamp_addi`, `severity_trending_datas`, `priority_data`, `sensor_datas`, `shutdown_periods`, `correlation_nowparam`, `correlate_sensor_datas`, `correlate_sensor_addi_datas`, `correlate_trending_datas`

Meaning

This is the deepest investigation payload in the application. It returns the selected signal, the severity trend, correlated signals, shutdown intervals, and priority information.

26. Settings Page and API

Page flow

`/settings/ -> pages.views.settings_page(request) -> render(request, "settings/settings.html", {})`

API flow

`/settings/ -> /api/adjust_threshold_settings -> apis.views.adjust_threshold_settings()`

Arguments and process

The view reads `start_date` and `end_date`, but there is no helper call yet and no threshold update logic yet.

Return

`{'datetime_last': ""}`

27. Final Summary Table

Page	API	Main process	Main return
/	`timeinfo_detail()`	latest timestamp lookup	time info
/	`severity_plot()`	select top anomaly traces	traces and prediction payload
/dashboard/	`panel_summary()`	summary maps and priorities	summary payload
/dashboard/	`zone_distribution()`	operation distribution	distribution payload
/kpi/	`unitStatus()`	alive/shutdown status	`status_dict`
/kpi/	`kpi()`	KPI, plant, manual KPI load	KPI payload
/kpi_units/	`zone_distributionTimeline()`	zone code timeline	timeline payload
/kpi_units/	`kpi()`	reused KPI payload	KPI payload
/advisory/	`advisory_table()`	severity ranking	advisory table payload
/advisory/chart	`advisory_detail()`	deep detail and correlation	advisory detail payload

28. Final Takeaway

Main conclusion

For pages with more than one API, each API should be read separately: arguments in, process inside helper, and payload out.

- Home, Dashboard, KPI, and KPI Unit pages each use two different APIs.
- Advisory and Settings use one API each.
- Manual upload is a special case because the page view itself handles the POST processing.